

Weave

A Graph-Native Programming Language

Multidimensional knowledge.

Graph-valued computation.

Explicit meaning across every composition.

VERSION 0.1 • 11 SEPTEMBER 2026

Prepared for Luis Palomo / Working title: Weave

ABSTRACT

Weave is a proposed programming language for AI agents and human developers to create, query, join, and revise multidimensional temporal knowledge graphs. A single entity can have linked manifestations in several graph spaces, each with its own state and geometry. Nodes and edges are first-class objects whose metadata may itself be a queryable graph. Parameterized graph-valued functions preserve evidence, temporal scope, identity mappings, and visibility through composition. The language exposes semantic zoom, portable offline branches, optional governance, and capability-scoped reactions to engine events. This paper defines the proposed semantic model, illustrative syntax, execution boundaries, interoperability approach, and conformance criteria. It separates design requirements from implementation choices and unresolved research questions.

A proposed design, not an implementation or a benchmark report.

Contents

The language defines meaning. The engine implements execution.

1. Purpose and design position	3
2. Core semantic model	3
3. Nodes and edges with graph-valued metadata	5
4. Types, schemas, and safe composition	6
5. Parameterized graph computation and temporal reasoning	8
6. Spaces, 3D directionality, and bridges	9
7. Automatic clustering and semantic zoom	9
8. Distribution, attachment, and offline branches	10
9. Permissions, visibility, and optional governance	10
10. Reactive programs and the engine bus	11
11. Compiler, tooling, and interoperability	12
12. Conformance, implementation sequence, and open questions	12
References	14

THREE DESIGN COMMITMENTS

IDENTITY One entity can have several linked manifestations; shared identity does not imply shared state.

COMPOSITION Nodes, edges, and metadata graphs retain their history, evidence, and boundaries.

EXECUTION Pure graph computation is separate from transactions, subscriptions, and external effects.

READERS Language designers, AI-system builders, graph-platform engineers, and technical reviewers.

COMPANION Paper 02 / Weave Engine

1. Purpose and design position

Weave is a proposed programming language for constructing, querying, composing, and revising multidimensional knowledge graphs. Its primary audience is AI agents and the people who build, inspect, and govern their work. It is also usable by ordinary programs: no language model, transformer, vector encoder, or remote AI service is required by the semantic core.

A Weave program does not merely retrieve records. It produces a graph together with the information needed to interpret that graph: selected revisions, temporal scope, identity mappings, evidence dependencies, visibility constraints, and coverage. That output can become the input to another program without discarding its meaning.

The language and its companion engine have different responsibilities. Weave specifies what an operation means and what effects it may request. The engine implements storage, transactions, event delivery, adapter execution, replication, and resource scheduling. The language must remain implementable by more than one engine, and the engine must expose the same semantics to clients that do not use Weave source code.

Design status. This is an architecture proposal, version 0.1, not a released language, a complete formal standard, or a report of measured results. Code samples illustrate a coherent proposed syntax; they have not been compiled by an implementation. “Must” identifies a proposed conformance requirement. “Should” identifies a preferred design choice. Performance and research questions are explicitly separated from those requirements.

1.1 What multidimensional means

A dimension is a graph space containing manifestations of entities. The same entity may have a physical manifestation, an operational manifestation, a semantic manifestation, and a private analytical manifestation. These manifestations refer to their shared identity and expose authorized links to their counterparts. They do not automatically share all properties, positions, or relationships.

This differs from attaching labels such as tenant or jurisdiction to one node. Context labels remain useful, but they are not the definition of a dimension. A branch is an alternative revision lineage; a replica is a stored copy; a space is a representation domain. All three are independent.

1.2 Intended contribution

The proposed contribution is a unified contract across graph-valued computation, linked manifestations, graph-valued metadata, temporal evidence, portable branches, and explicit effects. It is not a claim that graphs, provenance, recursive queries, or event-driven programming are new. RDF datasets already organize named graphs, and PROV-O supplies an established vocabulary for derivation and attribution. Weave proposes language-level composition rules over a broader runtime contract. [L1, L2]

2. Core semantic model

The logical store contains entities, manifestations, edges, assertions, graphs, metadata attachments, and revisions. Each has an addressable identity where persistence is required. A graph is a versioned selection of objects and assertions, not a promise that all referenced objects are locally present.

Object	Meaning	Identity rule
Entity	A particular thing or concept	Stable logical identity; not a content hash
Manifestation	An entity represented in a space	Own identity plus an authorized identity link
Edge	A typed relationship occurrence	Own identity; endpoints and predicate are structural
Assertion	Support, refutation, or assumption about knowledge	Source-specific, temporal, revisioned record
Graph	A queryable collection and its boundary	Logical graph ID plus immutable revision
Metadata attachment	A named value attached to an object	Independently versioned and attributable

An edge connects node manifestations by default. Its schema determines direction, endpoint types, and whether crossing spaces is allowed. Parallel edges are legal: two cables, two observations, or two relationships with different origins need not collapse into one. Reversing endpoints or changing a structural predicate creates a new edge identity rather than silently changing the meaning of an existing one. An optional n-ary relation form generalizes edges to named endpoints without weakening ordinary node-to-node edges.

Assertions separate a recorded relationship from endorsement of that relationship. Different sources can support or refute the same proposition; the engine retains their identities. Creating a structural edge does not establish external-world truth. A query specifies which accepted assertions, assumptions, and evidence policy it considers.

2.1 Identity across spaces

A manifestation references an identity-link set. The language exposes counterparts (node) as a virtual adjacency operation over that set, so every manifestation can navigate toward the other accessible manifestations of the same entity. An implementation may normalize membership rather than materialize every pairwise link.

Identity equivalence is accepted under a named policy and snapshot. A similarity score is not an identity assertion. An erroneous identity merge can be superseded or split while preserving history; historical queries continue to use the mapping revision they selected.

Private spaces may expose pairwise identifiers and protected mappings instead of one public correlating identifier. Counterpart discovery therefore reports coverage relative to authorized, available membership records. It must not promise knowledge of counterparts created on disconnected devices. W3C DID guidance provides relevant background on identifier correlation and pairwise identifiers. [L3]

2.2 Graphs with explicit boundaries

A graph snapshot comprises selected node, edge, and assertion identities; metadata bindings; and boundary references. A reference may resolve locally, through an authorized attachment, or not at all. Algorithms must distinguish a local induced subgraph from a graph with unresolved external endpoints. A remote endpoint is not silently replaced by a fabricated local node.

Graph membership is separate from ownership and identity. One edge or entity can participate in multiple views without becoming a new real-world object. Copying a graph snapshot preserves logical identities; cloning a hypothetical object requires an explicit identity-creation operation.

3. Nodes and edges with graph-valued metadata

Both nodes and edges support metadata. A metadata value can be a scalar, a typed vector, a collection, an artifact reference, an object reference, or a graph. Graph-valued metadata is a native queryable value, not an opaque JSON field and not only a URL to an external document.

For example, a device node can carry a maintenance-history graph. The edge connecting that device to a gateway can carry an evidence graph containing measurement nodes, calibration records, source passages, and the relationships that justify the connection. Nodes and edges inside either metadata graph can themselves carry scalar or graph-valued metadata.

ILLUSTRATIVE WEAVE

```
schema Infrastructure {
  node Device meta {
    label: String;
    maintenance: Graph<Maintenance>;
  }
  node Gateway;
  edge Connected(Device -> Gateway) meta {
    channel: String;
    evidence: Graph<Evidence>;
  }
}
```

Maintenance and Evidence denote separately declared graph schemas. The edge's evidence field is not less expressive than the node's maintenance field. Both support joins, temporal selection, traversal, branching, and access control.

3.1 Literal graphs, pinned references, and live handles

A graph literal constructs a graph value. Persisting that value creates a graph object and binds the metadata attachment to a revision. A pinned reference names an existing graph revision. Assigning it shares that snapshot; it does not deep-copy its contents.

ILLUSTRATIVE WEAVE

```
transaction Operations {
  let evidence = graph<Evidence> {
    node reading: Measurement { value: 12.4, unit: "ms" };
    node report: Document { key: "report-814" };
    edge source: RecordedIn(reading -> report);
  };
  set link17.meta.channel = "modbus";
  set link17.meta.evidence = evidence;
  set device17.meta.maintenance = pin(History, "h42");
}
```

Constructor fields such as value and unit are shorthand for schema-declared scalar metadata. The containing transaction atomically creates a new inline metadata graph and its attachment when they share a local transaction domain. Binding an already remote graph does not create a distributed transaction; it creates a reference with declared availability. A validator may require local materialization before accepting a particular schema.

An explicitly typed `LiveGraphRef<S>` can follow a graph head. It is distinct from immutable `Graph<S>`. A pure query must resolve such handles once into a snapshot manifest before evaluation. A live subscription resolves them at each emitted revision. An ordinary metadata read never silently tracks an advancing head during one evaluation.

Updating a shared metadata graph creates a new revision. Existing pinned attachments remain unchanged until explicitly rebound. A live binding may observe the update according to its declared policy. This makes shared evidence possible without hidden mutation of old results.

3.2 Metadata is contextual and attributable

Metadata attachments have their own assertion identity, valid interval, origin, and visibility. Entity-level metadata, manifestation-level metadata, and edge-level metadata are separate scopes. There is no implicit fallback that copies private identity-level fields into every manifestation.

A metadata graph is not automatically asserted as a conjunction by its host. An attachment named hypotheses may contain unresolved assumptions; one named evidence may contain competing accounts. Rules must explicitly state how they interpret and use it. Likewise, a user-created field named permission is ordinary data, not executable authorization policy.

3.3 Cycles and recursive depth

Graph references may form cycles, including a metadata graph referring back to its host. Traversal uses visited-object sets, revision-aware references, and explicit budgets. Serialization uses references rather than recursively embedding every reachable object. The language does not impose a fixed nesting depth, but every execution has finite resource limits.

Garbage collection follows reachability from retained graph and branch roots, including metadata attachments. Removing one attachment does not delete a shared graph still referenced elsewhere. Strong retained references and optional weak external references must be distinguishable by schema or storage policy.

3.4 Metadata query example

ILLUSTRATIVE WEAVE

```
from g = snapshot(Operations, at: checkpoint);
match (d: Device)-[e: Connected]->(gw: Gateway) in g;
from proof = e.meta.evidence;
match (m: Measurement)-[:RecordedIn]->(doc: Document)
  in proof;
where m.meta.value > threshold;
emit graph { include d, e, gw, m, doc; }
provenance auto;
```

The result retains the metadata attachment revision as a dependency. Authorization to read the edge alone does not authorize reading proof. If the proof is unavailable, the execution reports incomplete coverage or fails under a strict policy; it does not infer that the evidence graph is empty.

4. Types, schemas, and safe composition

Weave uses a small explicit grammar and a machine-readable abstract syntax tree. Natural language can help an agent produce programs, but executable meaning comes from typed syntax and versioned schemas.

The core value family includes booleans, integers, decimals, strings, bytes, timestamps, intervals, typed quantities, object references, graph values, and vectors. A graph type describes permitted node and edge schemas, space constraints, metadata shapes, and completeness requirements. Optional values distinguish a genuinely missing permitted field from a reference that cannot be resolved; restricted existence may deliberately remain indistinguishable to an unauthorized principal.

Node and edge references are different types. `Vec<3, BuildingA, Meter>` is not interchangeable with `Vec<1024, EncoderV7, Unitless>`. Schema evolution creates versioned migrations; a reader must not guess that a renamed field or changed unit has identical meaning.

4.1 Functions, lenses, and effects

A pure function maps ordinary values to values. A lens maps graph snapshots and parameters to a graph result. A transaction proposes writes to a declared local transaction domain. A reactive handler consumes an event and requests declared effects through capabilities.

ILLUSTRATIVE WEAVE

```
lens SiteEvidence(
  source: Graph<Infrastructure>,
  minimum: Decimal,
  checkpoint: Snapshot
) -> Graph<EvidenceView> effects { read(source) }
```

The effect annotation restricts accessible resources rather than introducing hidden network access. A pure lens cannot read the wall clock, invoke a model, issue a payment, or mutate a graph. Network resolution and nondeterministic computations are separate effects whose results must be recorded before reproducible graph evaluation.

The compiler checks static types, declared effects, visible schema compatibility, and statically known cross-space violations. The runtime checks capabilities, actual identities, temporal compatibility, quotas, and data-dependent constraints. Compilation is not proof that input claims are true or that an authorization grant is still current.

4.2 Graph algebra

union combines memberships while preserving assertion identities. match and join bind compatible objects. project changes representation using an explicit mapping. emit constructs a graph. resolve applies a named evidence or conflict policy. diff compares snapshots. These operators must not quietly substitute for one another.

Independent assertions of the same proposition remain independent evidence records. Repeated transport of one assertion remains one record. A join can create alternative derivations of the same conclusion, but provenance must preserve which premises jointly support a derivation and which derivations are alternatives.

A graph result is semantically closed under further composition: its schema, selected snapshots, assumptions, provenance, and boundary status accompany the graph. This does not mean the graph contains the transitive closure of every referenced object.

4.3 Joining an operational graph to a document graph

The following query crosses an edge's metadata graph, then joins a document from that evidence with an independently maintained catalog. The mapping is explicit: matching titles or nearby embeddings is not sufficient identity evidence.

ILLUSTRATIVE WEAVE

```
from ops = snapshot(Operations, at: inputs.operations);
from catalog = snapshot(Documents, at: inputs.documents);
match (d: Device)-[e: Connected]->(gw: Gateway) in ops;
from proof = e.meta.evidence;
match (m: Measurement)-[:RecordedIn]->(source: Document)
  in proof;
match (entry: Document)-[:ReviewedBy]->(reviewer: Agent)
  in catalog;
where same_entity(source, entry, using: inputs.identity_map);
emit graph { include d, e, gw, source, entry, reviewer; }
valid intersection()
provenance auto;
```

inputs pins each source and the accepted identity mapping. Documents supplies its own schema. The result records the match dependencies, including metadata bindings and the catalog relationship used by the join, even when only selected objects are projected into the output. A strict execution requires

the relevant proof and catalog scope to be available; a permissive execution marks the coverage it actually achieved.

5. Parameterized graph computation and temporal reasoning

Lens parameters include root entities, time windows, thresholds, task perspectives, and resource budgets. Parameters are not automatically new dimensions. Partial application creates reusable specialized lenses without copying their source.

ILLUSTRATIVE WEAVE

```
let FleetReview = SiteEvidence.bind(source: Fleet);
let september = FleetReview(
  minimum: 10.0,
  checkpoint: pin(Fleet)
);
```

For a simultaneous join, the output's valid interval is the intersection of the query window and the selected supporting assertions' intervals. If the intersection is empty, no simultaneous conclusion is emitted. A sequence query instead uses explicit temporal relations such as *before*, *overlaps*, or *within*; it must not inherit simultaneous semantics accidentally.

Valid time describes when an assertion claims to apply. A local recorded-time axis describes when a particular replica stored it. A causal checkpoint identifies revision ancestry. A governed view may add acceptance time. Bitemporal systems such as XTDB provide the valid-time/system-time precedent, but Weave must name the observing replica or accepted view rather than assume a universal recording clock. [L4]

ILLUSTRATIVE WEAVE

```
from knowledge = snapshot(Fleet,
  valid: ["2026-09-01", "2026-10-01"],
  known_at: replica("phone").checkpoint("c184")
);
```

Date-only literals in these examples denote midnight UTC, and intervals are half-open. Cross-peer queries pin a snapshot vector and required causal dependencies. Without an explicit coordination protocol, that vector is not advertised as a globally atomic snapshot.

5.1 Unknown, supported, refuted, and conflicted

For a selected proposition, context, and evidence policy, positive and negative support are tracked separately. No support is unknown; positive-only is supported; negative-only is refuted; both is conflicted. Unknown does not mean false, and a conflict must not entail arbitrary unrelated conclusions.

A closed-world operation is permitted only over a declared complete scope. Offline absence, hidden metadata, and a timed-out peer do not satisfy that condition. Conflict resolution creates an accepted interpretation without erasing the underlying disagreement.

Confidence annotations require a declared meaning, origin, and calibration status. An extraction score is not automatically a probability of truth. The language does not multiply arbitrary confidence numbers through graph joins.

5.2 Rules and provenance

The initial reasoning profile uses finite-domain, range-restricted rules, with stratified negation only over appropriately complete inputs. Unrestricted term generation and arithmetic recursion are excluded from the guaranteed-terminating profile. Souffle provides a relevant Datalog implementation precedent and documents how arithmetic extensions can permit nontermination. [L5]

A derivation references premise assertion IDs, relevant metadata attachment revisions, rule version, parameters, and input snapshots. Explanations are graphs. PROV-O can supply an interchange vocabulary, but a proof of derivation still establishes only that a conclusion follows from recorded premises under the selected rules. [L2]

6. Spaces, 3D directionality, and bridges

A space declares its schema, identity mapping policy, optional geometry, and applicable context. Geometry is optional: a commercial relationship graph need not invent coordinates.

ILLUSTRATIVE WEAVE

```
space Physical geometry Euclidean(dimensions: 3,
  frame: "building-A", units: Meter);
space Operations geometry None;
space Semantic geometry Embedding(dimensions: 1024,
  encoder: "encoder-v7", metric: Cosine);

manifest device17 in Physical as physical17;
manifest device17 in Operations as operational17;
manifest device17 in Semantic as semantic17;
```

Each manifestation carries its local representation while retaining an authorized counterpart relation. A physical position, an operational dependency, and a semantic neighborhood can change independently. Shared identity does not create an automatic state synchronization rule.

Spatial displacement, relation orientation, and embedding direction are different types of information. The vector from physical A to physical B is meaningful only in a common coordinate frame and compatible time interval. A depends_on edge remains directed regardless of its screen layout. A vector-valued edge annotation may express force, flow, orientation, or a semantic displacement only under its declared schema.

An identity bridge connects manifestations of the same entity. A semantic bridge relates different entities. A coordinate transform maps compatible frames. A learned mapping predicts a representation in another space and carries model version, direction, domain, and uncertainty. None of these implies the others. Equal vector length does not establish compatible embedding coordinates.

A 3D projection of a high-dimensional space is a navigational representation, not automatically the geometry used for authoritative similarity calculations. The language must name the original space and projection revision when exposing that view. Cross-space traversal explicitly records bridge dependencies in its result.

7. Automatic clustering and semantic zoom

Weave exposes clustering as a graph-valued computation with explicit perspective, inputs, visibility, time window, algorithm version, and budget. "Infinite" clustering means an unbounded schema of possible levels, created lazily; it does not require infinite storage, endless recursion, or invented detail beyond available evidence.

ILLUSTRATIVE WEAVE

```
let organization = clusterize(Operations,
  basis: [Topology, Semantic],
  overlap: allowed,
  depth: unbounded,
  visibility: current_principal,
  budget: nodes(5000)
);
let detail = zoom(organization, focus: device17,
  mode: semantic, budget: nodes(300));
```

A cluster is an addressable derived object. Its metadata graph records membership, supporting relations, source snapshots, quality measures, and split/merge lineage. Aggregate edges similarly retain graphs of contributing edges. Higher-level clustering operates on these derived representations while preserving access to originals.

The engine may offer overlapping organizations rather than one universal hierarchy. At a particular snapshot, membership edges designated as an abstraction hierarchy must not form cycles, even though ordinary graph edges may be cyclic. A navigational path can be tree-like without forcing global cluster membership into a tree.

Semantic zoom expands members, subclusters, relationships, or evidence. Geometric zoom only changes the camera. A summary used for approximate routing must be marked approximate; an exact query cannot exclude a cluster solely because a lossy summary appears irrelevant. Budgets and incomplete expansion are visible in the result manifest.

Clustering is an engine service accessed through the language, not a reserved implementation algorithm. Leiden supplies a community-detection precedent, but its network-community guarantees do not by themselves establish overlapping, temporal, privacy-aware clustering semantics. [L6]

8. Distribution, attachment, and offline branches

A capsule is a portable graph working set. It contains selected graph revisions, relevant metadata graphs, schemas, causal anchors, and a boundary manifest. Model artifacts and rule modules are included only when permitted and required. Explicit boundaries prevent a small offline selection from pretending to be a complete global graph.

The language distinguishes operations that are frequently conflated. `attach` makes a graph addressable. `replicate` copies authorized data. `fork` creates an independent lineage. `integrate` incorporates changes under a merge and acceptance policy. `detach` removes an active attachment. None of these automatically establishes identity equivalence or acceptance of external claims.

ILLUSTRATIVE WEAVE

```
let pocket = fork(Fleet,
  root: device17,
  include_metadata: required,
  dependencies: explicit,
  budget: storage(150.MB)
);
detach(pocket);
// Local transactions and lenses continue on the pocket branch.
attach(pocket, to: Team, mode: propose_changes);
```

A branch may span several spaces. An offline device keeps its own events, assertions, and metadata revisions. Reconnection exchanges authorized history before applying domain-level acceptance. A conflicting metadata graph attachment is preserved as a conflict between graph references; contents are not automatically blended merely because both values have graph types.

Local-first work is a design foundation, not a requirement that every device hold the entire graph. The local-first research literature motivates independent local operation and later collaboration. [L7] A query over an offline capsule reports its exact working-set boundary and unresolved dependencies. Degraded execution is explicit rather than silently weakening a rule's assumptions.

9. Permissions, visibility, and optional governance

The language exposes capabilities to discover, read, traverse, propose, publish, subscribe, execute effects, and delegate. Capabilities are scoped to resources and operations; a tenant label is not itself a grant. A reference or content identifier is not proof of access.

Node metadata, edge metadata, counterpart links, and graph boundaries are individually subject to policy. The effective right to traverse into a metadata graph must satisfy the host-path restrictions, attachment restrictions, and target-graph restrictions. The target graph's independent direct access policy remains separate; attaching a public graph to a private node does not make every other reference to that graph private.

Derived outputs inherit restrictions from the evidence that influenced that derivation unless an authorized release policy applies. This includes cluster labels, aggregate counts, coordinates, cached query results, and event envelopes. The presence of a restricted counterpart must not leak through ordinary errors or global cluster layout.

Authorization is not governance. Governance determines which proposals become part of an accepted view. A personal graph can use owner authority without collective governance. A shared graph may require review, threshold approvals, or a scoped ordering service for exclusive decisions. Policy changes are versioned and authorized under the preceding policy; a graph containing proposed rules cannot authorize its own installation.

Offline capabilities have a declared validity and acceptance policy. A disconnected replica cannot immediately learn a revocation, and software cannot guarantee recall of plaintext already disclosed. Weave therefore distinguishes local proposal creation from authoritative acceptance requiring current policy validation. UCAN is a relevant capability-delegation precedent, not a mandatory dependency or a substitute for these limits. [L8]

10. Reactive programs and the engine bus

The engine publishes events when committed knowledge, attachments, accepted views, or operational states change. Weave can declare subscriptions and reactive handlers, but event routing and adapter lifecycle belong to the engine.

A command requests an action and may fail. An event records an occurrence under a defined producer contract. A registered adapter consumes permitted events and submits commands through capabilities; it never bypasses the kernel by mutating storage directly.

ILLUSTRATIVE WEAVE

```
reactor IndexEvidence {
  on engine.MetaGraphRebound.v1 as event;
  requires capability("evidence-indexer");
  dedupe key(event.id, self.version, self.config_revision);
  effects { read(event.snapshot), propose(Semantic) }
  run {
    let graph = resolve(event.data.after_graph);
    let proposal = build_index_projection(graph);
    submit(proposal, to: Semantic);
  }
}
```

`engine.MetaGraphRebound.v1` is a registry-resolved language alias for the wire event type `org.weave.meta.graph-rebound.v1`; `event.snapshot` resolves its authenticated commit context. `build_index_projection` denotes an application-defined operation. A deterministic implementation can be replayed from pinned inputs. A model-backed implementation must record its model, parameters, actual output, and artifact revisions. Replay cannot silently call a changed model and claim to reproduce history.

Subscriptions identify event schema versions, resource scope, start checkpoint, delivery requirements, and replay mode. Durable delivery is at least once within declared retention and availability conditions; handlers must tolerate duplicates. Idempotent local commands use stable operation keys. External effects require an explicit effect ledger and a reconciliation policy for uncertain outcomes. No language construct promises exactly-once execution against arbitrary external systems.

Post-commit handlers cannot veto an already committed operation. Preconditions and policy checks execute within the validation path. External actions require their own authorization and may require human approval. A graph field containing code remains inert data unless installed as an executable module by an authorized principal.

CloudEvents provides an existing event-envelope vocabulary, including event identity, source, type, and schema information. Weave uses that as an interoperability baseline; delivery, authorization, causal order, and transactional publication remain additional engine contracts. [L9]

11. Compiler, tooling, and interoperability

The compilation pipeline is source parsing, name resolution, schema/type checking, effect checking, graph-plan construction, and lowering into a versioned intermediate representation. The IR carries explicit space mappings, temporal operators, provenance obligations, capability scopes, and completeness requirements. An engine must reject unsupported IR features rather than reinterpret them.

A plan fingerprint identifies the normalized program, parameters, schemas, and operator versions. A result fingerprint additionally includes selected graph revisions, visibility-policy context, identity mappings, and any model artifacts. Hashing does not itself establish semantic equivalence. Exact reproducibility requires pinned inputs and deterministic operators; floating-point or approximate operators declare their reproducibility class.

AI-oriented tooling includes schema discovery, typed completion, dry-run plans, machine-readable diagnostics, minimal failing examples, capability summaries, and provenance inspection. A diagnostic should identify the incompatible space or missing metadata dependency rather than merely return “invalid query.” Generated writes first form proposals that can be inspected and validated.

RDF export can represent graphs and edge identities through named resources and explicit modeling conventions. A round-trip manifest must declare unsupported or lossy mappings rather than claim universal equivalence. PROV-O export can represent selected provenance structures. Foreign-function calls and adapters are the extension boundary for existing databases, transports, and models. [L1, L2]

12. Conformance, implementation sequence, and open questions

The first implementation should prioritize semantic correctness over broad syntax. A minimum useful profile includes typed node and edge identities, pinned graph-valued metadata, local temporal snapshots, graph-valued lenses, explicit space bridges, capability checks, transactions, and a durable event interface. Offline branching and reconciliation follow on the same object contract; automatic clustering must not require a second incompatible data model.

Test	Required outcome
Edge evidence points to a graph	That graph can be queried and joined natively
A metadata graph refers to its host	Traversal terminates or returns an explicit budget limit
A shared metadata graph advances	Pinned attachments retain their previous meaning
Two peers edit one attachment concurrently	Both versions remain inspectable under conflict policy
Two manifestations share identity	Local state remains separate unless propagation is declared
Join premises never overlap in valid time	No simultaneous conclusion is emitted
An offline reference is missing	No closed-world negative is silently inferred
An adapter receives an event twice	A declared idempotent command has one logical effect
An input is private	No public derivative is emitted without release authority

The larger test suite should include property-based graph generation, adversarial cyclic metadata, snapshot/replay equivalence, cross-engine IR compatibility, and metamorphic tests showing that repeated transport does not create additional evidence. Benchmarks should measure query latency, provenance expansion, memory, metadata fan-out, and agent repair success. This paper reports no benchmark results.

Open questions include the ergonomics of graph-type variance, schema migration across disconnected branches, identity splits affecting historical derivations, bounded provenance representations, and formal noninterference for derived views. An eventual specification needs a complete grammar, an operational semantics, a canonical encoding, and a versioned conformance suite. None should be replaced by claims that a plausible syntax is already a working language.

Conclusion. Weave's defining property is graph-valued composition with retained meaning. Entities can have many manifestations; nodes and edges can carry further knowledge graphs; programs can move across representations and scales without confusing identity, evidence, authorization, or agreement. The companion engine turns those semantics into local and distributed execution through a durable, capability-controlled event bus.

References

- [L1] W3C. *RDF 1.1 Concepts and Abstract Syntax*. Recommendation, 2014. [Source](#)
- [L2] W3C. *PROV-O: The PROV Ontology*. Recommendation, 2013. [Source](#)
- [L3] W3C. *Decentralized Identifiers (DIDs) v1.0*, privacy considerations. [Source](#)
- [L4] XTDB. *Time in XTDB*. Official documentation. [Source](#)
- [L5] Souffle project. *Tutorial*. Official language documentation. [Source](#)
- [L6] Traag, V. A.; Waltman, L.; van Eck, N. J. *From Louvain to Leiden: guaranteeing well-connected communities*. 2019; preprint arXiv:1810.08473. [Source](#)
- [L7] Kleppmann, M.; Wiggins, A.; van Hardenberg, P.; McGranaghan, M. *Local-first software: You own your data, in spite of the cloud*. Onward!, 2019. [Source](#)
- [L8] UCAN working group. *User Controlled Authorization Network specification*. Living specification. [Source](#)
- [L9] CloudEvents project. *CloudEvents specification*, version 1.0.2. [Source](#)

All online references were consulted on 11 September 2026. They establish related foundations, not validation or endorsement of Weave. The companion document is *Weave Engine: An Event-Driven, Decentralized Knowledge Runtime*, architecture proposal v0.1.